

Program 3: Code a Dockerized python Flask or Node.js Application

Project Structure

program-3

-----> Dockerfile
-----> requirements.txt
-----> app.py

step 1: create the Dockerfile and add the contents
nano Dockerfile

```
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx: ~/prg3
GNU nano 6.2 Dockerfile
# Use official Python image
FROM python:3.9-slim

# Set working directory
WORKDIR /app

# Copy app files
COPY . /app

# Install Flask
RUN pip install flask

# Expose port 5000
EXPOSE 5000

# Run the app
CMD ["python", "app.py"]
```

step 2: create the file app.py
nano app.py

```
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx: ~/prg3
GNU nano 6.2 app.py
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from Simple Flask Docker!"

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

step 3: create the requirements.txt
nano requirements.txt

```
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx: ~/prg3
GNU nano 6.2 requirements.txt
Flask==2.3.3
```

step4: Execute the Docker Build Command
docker build -t prg3

```
bash: /home/pavithra/dev/flutter/bin/cache: No such file or directory
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~$ mkdir prg3
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~$ cd prg3
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/prg3$ nano Dockerfile
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/prg3$ nano requirements.txt
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/prg3$ nano Dockerfile
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/prg3$ nano app.py
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/prg3$ nano Dockerfile
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/prg3$ docker build -t prg3 .
[+] Building 208.4s (10/10) FINISHED
=> [internal] load build definition from Dockerfile
=> => transferring dockerfile: 344B
=> [internal] load metadata for docker.io/library/python:latest
=> [internal] load .dockerignore
=> => transferring context: 2B
=> [internal] load build context
=> => transferring context: 617B
=> [1/5] FROM docker.io/library/python:latest@sha256:a8053dece0fb59fd31e0eb615302e1d3936191655bc8ec74db128107fa415a08
=> => resolve docker.io/library/python:latest@sha256:a8053dece0fb59fd31e0eb615302e1d3936191655bc8ec74db128107fa415a08
=> => sha256:a8053dece0fb59fd31e0eb615302e1d3936191655bc8ec74db128107fa415a08 10.95kB / 10.95kB
=> => sha256:0cb593dd6b3a67de3166f978b87cd8981942a1200a21b29d5385334e4b99384 2.32kB / 2.32kB
=> => sha256:e396456a47e83b4786e3303c14e1a19ebf3929ec633134ff837493a89301e8ea 6.38kB / 6.38kB
=> => sha256:795d8bede24d2c72dafd2b71fe36643552e56859c0e29c0b095ed54b825fbaa2 49.28MB / 49.28MB
=> => sha256:89d573bf42b377ce6a5a0451c15388849686fa4058ef68999f3b014dae5b55 25.62MB / 25.62MB
=> => sha256:26dfe2fac1c486e9aaf41d1028ed30be2c442aa84af44462bc7bac8c148ffb13 67.78MB / 67.78MB
=> => sha256:79d5bd8a8d262418bf22e705535ce38c6789dc72e319d76b30aafa5c331b6924 235.93MB / 235.93MB
=> => sha256:31ecb0fa272dab2687d1e1423ffa4a44463d14bd76ddacea180c9dcdf7817b7b 6.09MB / 6.09MB
=> => extracting sha256:795d8bede24d2c72dafd2b71fe36643552e56859c0e29c0b095ed54b825fbaa2 3.0s
=> => extracting sha256:89d573bf42b377ce6a5a0451c15388849686fa4058ef68999f3b014dae5b55 1.2s
=> => sha256:444728a573587a84f12124af83e21c91c9d9258124d8a6d63c9c1888efc119a7 29.23MB / 29.23MB
=> => extracting sha256:26dfe2fac1c486e9aaf41d1028ed30be2c442aa84af44462bc7bac8c148ffb13 4.4s
=> => sha256:6287f334c0e7561c0eb63587c59927ba24855a3e29a23bd5b32d5842af55cd68 249B / 249B
=> => extracting sha256:79d5bd8a8d262418bf22e705535ce38c6789dc72e319d76b30aafa5c331b6924 13.7s
=> => extracting sha256:31ecb0fa272dab2687d1e1423ffa4a44463d14bd76ddacea180c9dcdf7817b7b 0.4s
=> => extracting sha256:444728a573587a84f12124af83e21c91c9d9258124d8a6d63c9c1888efc119a7 1.2s
=> => extracting sha256:6287f334c0e7561c0eb63587c59927ba24855a3e29a23bd5b32d5842af55cd68 0.0s
=> [2/5] WORKDIR /app
=> [3/5] COPY requirements.txt .
=> [4/5] RUN pip install --no-cache-dir -r requirements.txt
=> [5/5] COPY . .
=> exporting to image
=> => exporting layers
=> => writing image sha256:1457e74fca3cc0917932bc6a7c6bf3e3804cec9e187bd790baf27ec069d85ae1
=> => naming to docker.io/library/prg3
```

step 5: run the docker command specifying the port number

```

1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/prg3$ docker run -d -p 5000:5000 prg3
1148ac93c721c3f40349156209bdd8d4085e4abfef563475cf4b79b9bde2f9
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/prg3$ docker ps
CONTAINER ID   IMAGE      COMMAND                  CREATED        STATUS        PORTS                               NAMES
1148ac93c721   prg3       "python app.py"         32 seconds ago Up 31 seconds  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp  boring_greider
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/prg3$ docker logs <1148ac93c721>
bash: syntax error near unexpected token `newline'
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/prg3$ docker logs 1148
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on all addresses (0.0.0.0)
* Running on http://127.0.0.1:5000
* Running on http://172.17.0.2:5000
Press CTRL+C to quit
1RV24MC073_pavithra@pavithra-HP-Laptop-15-db1xxx:~/prg3$ docker exec -it 114 bash
curl http://localhost:5000
root@1148ac93c721:/app#

```

step 6: verifying in the localhost

